

Deliverable report

“Alessio Blascovich”: Mat: 257561, alessio.blascovich@studenti.unitn.it, *GitRepo*:
<https://github.com/elblasco/GPU-Computing-2025-257561>

Abstract—The sparse matrix-dense vector multiplication (SpMV) is an algebraic operation in which a sparse matrix is multiplied by a dense vector. SpMV is widely used in real-world applications such as scientific and engineering computing, data mining, and graph analysis.

This deliverable discusses the implementation and the performance analysis of a series of SpMV algorithms for COO matrices. These algorithms are implemented for a single-core CPU and a CUDA capable GPU. Regarding the GPU implementations, no threads/warp level synchronisation has been used, only atomic operations to keep the resulting vector consistent.

Index Terms—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

I. INTRODUCTION

SpMV is considered a fundamental algebraic operation in many scientific fields such as computational fluid dynamics, optimisation and graph algorithms.

The main challenges in implementing an efficient SpMV algorithm differ in CPU and GPU, and are strictly bound to the type of matrix representation used (COO, CSR, ELL,...). For the purpose of this deliverable only COO representation is taken into account.

From the CPU point of view, the main issue is to maximise the cache hits throughout the execution, data consistency is not a problem since only single core CPUs were used.

On the GPU side the problem of maximising cache hits remains, and on top of that, race conditions are added. Since in COO format the length of a row is not specified, multiple threads could be reading different portions of the same row and therefore, writing on the same output memory area.

II. PROBLEM STATEMENT

The SpMV operation is formally defined in Equation (1), where A is a $n \times m$ sparse matrix, X a $m \times 1$ dense vector and Y another $n \times 1$ dense vector.

$$Y = A \cdot X \quad (1)$$

A. Storage Format

The COOrdinate format (COO) is a representation for sparse matrices. COO uses 3 different arrays to store all the necessary information. One array dedicated to store all the values, while the other 2 contain the corresponding row and column indices. In order to parse the files I used a fork of `distributed_mmio`¹.

¹The fork author is Matteo Di Noia https://github.com/matteo-dinoia/distributed_mmio while I just fixed a minor issue https://github.com/elblasco/distributed_mmio

B. Parallelization

2 different kernels have been used for the CUDA capable GPU, although in both of them the workload is divided equally among all the threads used. The main difference is the pattern used to access the global memory from a thread. In the first approach each thread jumps of stride equivalent to the number of threads used, while in the second approach a thread access to a continuous area of memory long as the total number of non-zero element in the matrix divided by the number of threads used.

III. STATE OF THE ART

Starting from the implementations presented in this deliverable, performances could be drastically enhanced. Fukaya et al. [1], despite focussing on the HDC storage format, improved the SpMV CPU based by exploiting diagonalization patterns, SIMD instructions and *OpenMP* directives. On the GPU side Chu et al. [2] used the CSR format and a more sophisticated load balancing to improve the performances. NVIDIA itself made available a set of libraries to manage and use COO formatted matrices, namely *cuSPARSE*.

In the NVIDIA library *cuSPARSE*, and many others commercial libraries, the problem is redefined according to Equation (2).

$$Y = \alpha op(A) \cdot X + \beta Y \quad (2)$$

With α and β being two scalars, and *op* being a matrix transformation such as a transposition.

IV. METHODOLOGY AND CONTRIBUTIONS

The tests has been carried out performing multiple execution rounds. The execution has been split into two stages, a warm-up period and a “valid” period, the data has been collected only during the “valid” period.

A. GPU

For the GPU implementation I have developed four kernels, they are divided into two main categories. Kernel that leverage shared memory and kernels that leverage warp level primitives. Algorithm 1 was the first kernel developed using warp-level primitives. Every thread proceed with a striding of *numThreads* values, initially every thread computes the product between its the matrix value and the value for the corresponding dense array. This values are then combined using a prefix-sum based on warp-level instructions.

Algorithm 2 is just a prefix sum that exploits warp instructions and take into account the possibility that within the same

Algorithm 1 SpMV algorithm with WARP level primitives

Require: The input vectors val , row and col of size nnz , array arr of size $columns$ and an array res of size $rows$.

```

1: procedure FUNCTION( $row, col, val, arr, res, \dots$ )
2:    $start \leftarrow threadId$ 
3:    $pos \leftarrow threadId \& (WARP\_SIZE - 1)$ 
4:    $total \leftarrow gridDim \cdot blockDim$ 
5:   for  $i$  in  $\{start \dots nnz\}$  with  $i = i + total$  do
6:      $row\_id \leftarrow row[i]$ 
7:      $col\_id \leftarrow col[i]$ 
8:      $val\_id \leftarrow val[i]$ 
9:     //The last thread per row writes
10:     $row\_next \leftarrow shfl\_down(0xff \dots, row, 1)$ 
11:    if  $Edge\_detection()$  then
12:       $atomicAdd(\&res[row], prod);$ 
13:    end if
14:  end for
15: end procedure

```

Algorithm 2 Prefix-sum with WARP level primitives

Require: The values for $product$, row and pos .

```

1: procedure PREFIX_SUM_WARP( $product, row, pos$ )
2:   for  $x$  in  $\{1 \dots WARP\_SIZE\}$  with  $x = x \cdot 2$  do
3:      $prev\_row \leftarrow shfl\_up(\dots, row, x)$ 
4:      $prev\_prod \leftarrow shfl\_up(\dots, product, x)$ 
5:     if  $pos \geq delta \wedge prev\_row = row$  then
6:        $product = product + rev\_prod$ 
7:     end if
8:   end for
9: end procedure

```

warp we could be summing different rows. In Figure 1 it is described a possible execution for the prefix sum procedure, in red the value returned summed in global memory in line 12 of Algorithm 1. The next improvement introduced was the loop unroll of the main loop in Algorithm 2, using compiler directives. To be precise, I used `#pragma unroll`, since the loop always iterates in an interval known at compile time, as a matter of fact in my code `WARP_SIZE` is a macro that define the value 32.

I also tried to improve the baseline proposed in deliverable 1 by using dynamically allocated shared memory.

V. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP

A. System Description

For this deliverable I could not use my own machine since it does not have a *CUDA* capable GPU, so I only used the remote cluster. Table I describes all the software used to set-up the environment and run the kernels.

B. Dataset description

To add a bit more of generality to the dataset I picked six matrix from six different kinds from the site <https://sparse.tamu.edu/>.

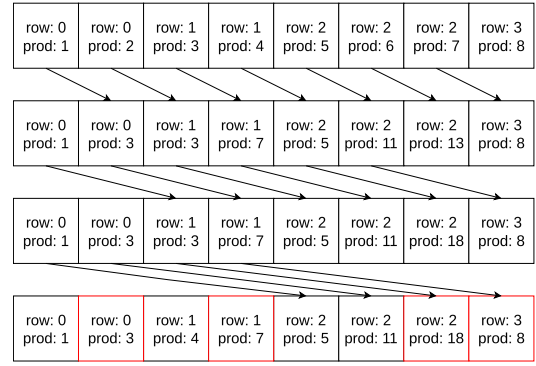


Fig. 1. Execution of prefix sum

Software	Version	Git Branch	Git Hash
GCC	11.5.0		
CUDA	12.5.0		
SbatchMan		example	40d869a
distributed_mmio		main	3563a2f

TABLE I
SYSTEMS SOFTWARES

tamu.edu/. Since all the matrices are squared only the number is written on Table III.

- 1) Electromagnetics Problem;
- 2) Undirected Weighted Graph;
- 3) Optimization Problem;
- 4) Structural Problem;
- 5) 2D/3D Problem;
- 6) Semiconductor Process Problem.

C. Experimental Set-up

I used 5 warm-up runs followed by 5 “valid” runs to collect data. I kept this set-up even though after few tests one warm-up round was enough to have stable values, as a matter of fact I decided to keep all the data generated. Each job submitted to the cluster executed a full warm-up and run for Algorithm ??, then Algorithm ?? and then Algorithm ??.

VI. EXPERIMENTAL RESULTS

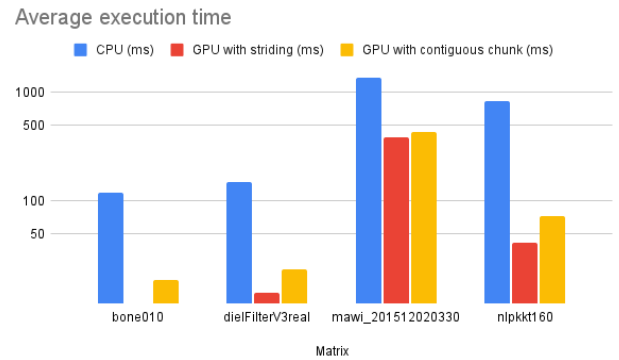


Fig. 2. Average Execution Time (lower is better)

System	Processor	Cores per Socket	RAM	Accelerator
Cluster	Intel(R) Xeon(R) Silver 4214	48 at 2.2 GHz	400 GB	NVidia L40s

TABLE II
SYSTEMS DETAILS

Name	Rows	Pattern Entries	Symmetric	k
dielFilterV3real	1,102,824	89,306,020	Yes	
Flan_1565	1,564,794	117,406,044	Yes	
mawi_201512020330	226,196,185	480,047,894	Yes	
nlpkkt160	8,345,600	229,518,112	Yes	
Queen_4147	4,147,110	329,499,284	Yes	
vas_stokes_2M	2,146,677	65,129,037	No	

TABLE III
INPUT MATRICES

As expected in Section IV and shown in Figure [2, 3], Algorithm ?? is faster than all the other implementations.

I decided to dig a bit more into the performances of Algorithm ?? by changing the maximum amount of thread per block. Starting from the base of 512 threads I divided by 2 until reaching 32, i.e., the size of warp. The optimal amount of threads per block resulted to be 256. This is due to the use of an atomic instruction within the code, the semantic of the instruction make concurrent write over the same address sequential; in this case if two threads are working on the same row their sum operations will be sequentialised since they are trying to write on the same result address. Using less threads, i.e., each thread works on more elements; conflicts (threads working) on the same row are reduced to a minimum. However, this minimum is not stable, in fact decreasing again the value to 128, 64 or 32 the performances decrease again, this could be symptom of either:

- too few threads working, so an excessive workload for each of them;
- too much threads working on the same row, hence an high number of conflicts on the same row.

Unfortunately, I was not able to determine a more fine grain optimisation, since the optimal size depends on the input matrix shape. Figure [4, 5] highlights the fact that the variations of performances are not constant over all the matrices considered.

Following the NVIDIA's white-paper for the A30, I drew the roofline model for double precision floating point, since in my code I used `double` as data type. Figure 6 highlights the fact that my `CUDA` kernels are indeed a memory bounded.

VII. CONCLUSIONS

The presented algorithms are far from decent implementations. The representation used for the matrices is sub-optimal since the length of a row is unknown a priori, therefore it is impossible to do a decent load balancing between the threads. For better GPU performances using another representation such as CSR would work better.

Another bottleneck for the algorithm is represented by the use of an atomic instruction. Instead of this instruction thread/warp level synchronisation should be used.

Average GFLOP/s developed

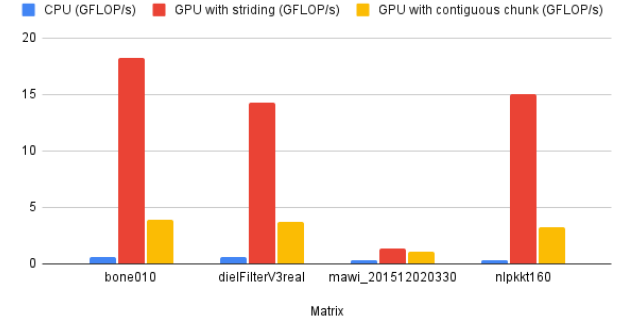


Fig. 3. Average GFLOP/s Developed (lower is better)

GFLOP/s changing threads per block

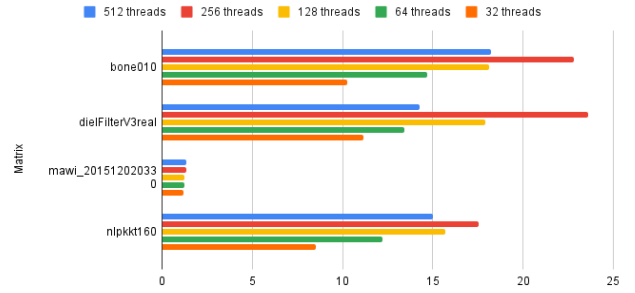


Fig. 4. GFLOP Compared The Number Of Threads Per Block

On the CPU side, this representation is optimal since it permits to maximise the number of cache hits working on data stored so closely.

REFERENCES

- [1] T. Fukaya, K. Ishida, A. Miura, T. Iwashita, and H. Nakashima, "Accelerating the spmv kernel on standard cpus by exploiting the partially diagonal structures," 2021. [Online]. Available: <https://arxiv.org/abs/2105.04937>
- [2] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing spmv on gpu," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 115–128. [Online]. Available: <https://doi.org/10.1145/3588195.3593002>

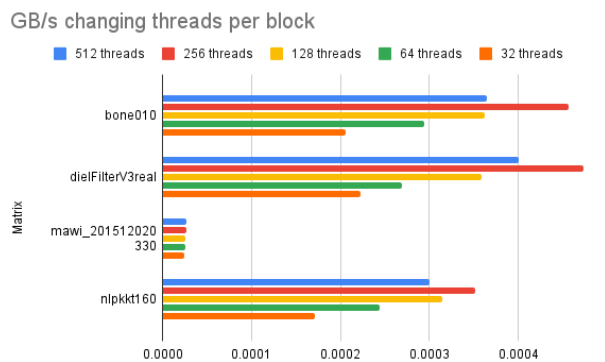


Fig. 5. GB/s Compared The Number Of Threads Per Block

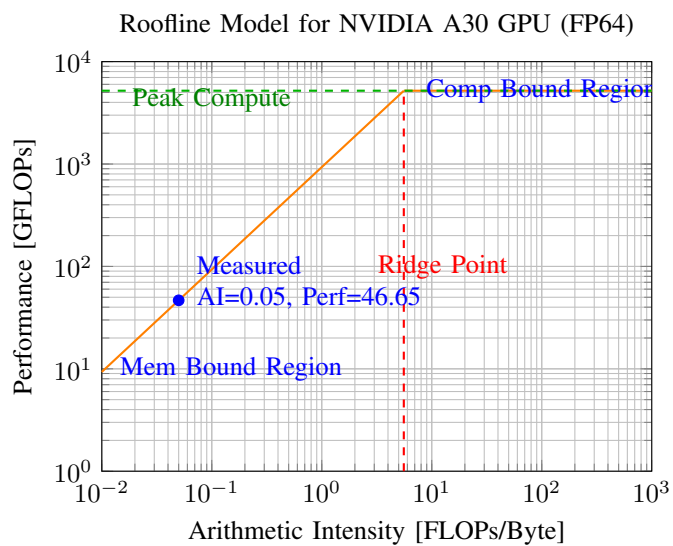


Fig. 6. Roofline Model For SpMV On NVIDIA A30